BASAVARAJESWARI GROUP OF INSTITUTIONS
BALLARI INSTITUTE OF TECHNOLOGY & MANAGEMENT
NBA and NACC Accredited Institution*
(Recognized by Govt. of Karnataka, approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University, Belagavi)" Jnana Gangotri" Campus, No.873/2, Ballari-Hosapete Road, Allipur, Ballari-583104 (Karnataka) (India)
Ph:08392 – 237100 / 237190, Fax: 08392 – 237197

DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

A

Project Report

On

"INTELLIGENT TRANSPORTATION SYSTEM USING DEEP LEARNING"

A dissertation submitted to the Department of Artificial Intelligence and Machine Learning of Visvesvaraya Technological University in partial fulfillment for the Degree of Bachelor of Engineering award.

Submitted By

G ESHWAR **3BR23AI405**

Under the Guidance of
Dr. B.M Vidyavathi
Professor & HOD
Dept of AIML, BITM, Ballari

Visvesvaraya Technological University
Belagavi, Karnataka
2025-2026

BASAVARAJESWARI GROUP OF INSTITUTIONS
BALLARI INSTITUTE OF TECHNOLOGY & MANAGEMENT
NBA and NACC Accredited Institution*
(Recognized by Govt. of Karnataka, approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University, Belagavi) "Jnana Gangotri" Campus, No.873/2, Ballari-Hosapete Road, Allipur, Ballari-583 104 (Karnataka) (India)
Ph:08392 – 237100 / 237190, Fax: 08392 – 237197

DEPARTMENT
OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

Certified that the project work entitled **"INTELLIGENT TRANSPORTATION SYSTEM USING DEEP LEARNING"** carried out by **Mr. G ESHWAR** bearing USN **3BR23AI405** a bonafide student of Ballari Institute of Technology and Management in partial fulfillment for the award of Bachelor of Engineering in Artificial Intelligence and Machine Learning of the Visvesvaraya Technological University, Belgaum during the year 2025-2026. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said Degree.

Signature of guide	Signature of HOD	Signature of Principal
Dr. B M Vidyavathi	**Dr. B M Vidyavathi**	**Dr. Yadavalli Basavaraj**

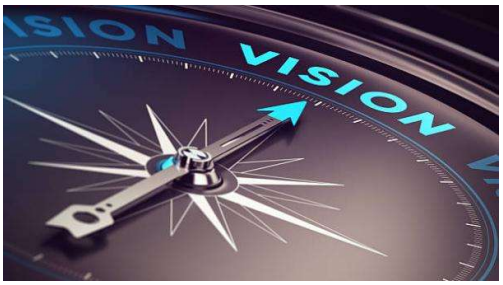
EXTERNAL VIVA

Name of the Examiners	Signature and date
1. ……..	……..
2. ……..	……..

BASAVARAJESWARI GROUP OF INSTITUTIONS
BALLARI INSTITUTE OF TECHNOLOGY & MANAGEMENT
NBA and NACC Accredited Institution*
(Recognized by Govt. of Karnataka, approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University, Belagavi)"
Jnana Gangotri" Campus, No.873/2,
Ballari-Hosapete Road, Allipur, Ballari-583104 (Karnataka)
(India) Ph:08392 – 237100 / 237190, Fax: 08392 – 237197

DEPARTMENT
OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VISION



To be a state-of-the-art learning center that responds to future technological challenges and drives innovations.

MISSION

M1: To impart strong foundation of theoretical & practical concepts.

M2: To equip with design and development skills catering to the MISSION needs of the industry and society.

M3 To promote research, entrepreneurship and leadership skills with ethical responsibilities.

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of project work on **"INTELLIGENT TRANSPORTATION SYSTEM USING DEEP LEARNING"** would be incomplete without the mention of people who made it possible, whose noble gesture, affection, guidance, encouragement, and support crowned my efforts with success. It is my privilege to express my gratitude and respect to all those who inspired me in the completion of this project work.

I am extremely grateful to my respective guide **Dr. B M Vidyavathi**, Professor and HOD of AI&ML DEPT., BITM, Ballari and my project Co-Ordinator **Mrs. P Asha Joythi**, Assistant Professor, AIML, BITM, Ballari and **Dr. B M Vidyavathi**, Professor and HOD of AI&ML DEPT., BITM Ballari for their noble gesture, support, coordination and valuable suggestions given to me in completing the project work. I also thank the Principal, Management, and non-teaching staff for their coordination and valuable suggestions given to me in completing the project work.

NAME	USN
G ESHWAR	3BR23AI405

ABSTRACT

An Intelligent Transportation System (ITS) powered by deep learning is designed to enhance road safety, traffic efficiency, and transportation management through automated decision-making. It collects real-time traffic data from sources such as CCTV cameras, sensors, and GPS devices. To assist users—including traffic operators and government authorities—an intuitive interface provides step-by-step guidance through a rule-based chatbot. After uploading or streaming traffic data, the system displays processed visual frames clearly, allowing users to select specific time periods, locations, or events they want analyzed. This frame-wise selection makes the system flexible and highly customizable. The main objective is to convert raw traffic footage into meaningful insights for improved transportation planning. Overall, the ITS modernizes urban mobility by transforming real-time road data into actionable intelligence.

Once the user selects the required video segment or dataset, the system begins by extracting features using advanced deep learning models such as CNNs, LSTMs, and object detection networks. These models identify vehicles, pedestrians, lane markings, and traffic violations with high precision. The extracted information is then processed through analytics modules to detect congestion, predict traffic flow, and classify incidents. Based on this analysis, the system generates concise, meaningful reports using AI summarization. Simultaneously, visual dashboards and annotated video frames are created to provide clear representations of traffic behavior. These insights and visuals are combined into an organized analytics summary, reducing manual surveillance effort and speeding up decision-making. As a result, traffic authorities receive structured, accurate, and easy-to-interpret outputs that support effective management

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

Transportation plays a crucial role in ensuring the smooth functioning of educational institutions, especially colleges where thousands of students and faculty depend on campus buses every day. Traditionally, college bus systems operate without real-time information, leaving students uncertain about bus arrival times, delays, or route changes. This often leads to long waiting periods, missed buses, overcrowding at stops, and overall inconvenience. With increasing campus sizes and busy schedules, the need for a reliable, technology-driven transportation solution has become more important than ever. Recent advancements in GPS technology, IoT devices, and Deep Learning algorithms have opened up exciting possibilities for improving transportation efficiency. Modern intelligent systems can continuously collect bus location data, analyse movement patterns, predict arrival times, and instantly notify users about delays. Such smart solutions are already being adopted by public transportation sectors worldwide, demonstrating their effectiveness in enhancing user experience and operational management.

Recognizing the growing need for a smarter and more organized campus transportation system, this project-"**NAMMA BUS: Intelligent Transportation System Using Deep Learning"** aims to integrate real-time tracking, predictive analytics, and automated notifications into a unified platform. By leveraging GPS-enabled IoT modules installed in college buses and a mobile/web application for students, the system provides accurate live tracking and Estimated Time of Arrival (ETA) predictions. Additionally, the admin dashboard assists management in monitoring routes, addressing emergencies, and improving service reliability. Overall, the project serves as a step toward adopting smart mobility solutions within college campuses, ensuring a safer, faster, and more efficient commuting experience through the use of modern intelligent technologies.

1.2 Problem Context

College transportation plays a crucial role in helping students and staff reach the campus on time, yet most institutions still depend on traditional, manual systems for managing bus operations. In the current setup, students rely on fixed schedules without having any real-time information about the actual bus location or delays. When buses are late due to traffic, weather, roadblocks, or breakdowns, students are left waiting at bus stops with no updates. This often leads to missed buses, wasted time, and increased frustration.

Transport administrators also struggle due to the lack of a centralized system. They have no real-time visibility of bus movement, making it difficult to monitor routes, ensure driver accountability, or respond to emergencies. Drivers may face unexpected route changes or traffic conditions but have no proper channel to communicate delays to students or administrators.

Although some tracking systems exist, they are either limited to simple GPS tracking or lack intelligent features such as ETA prediction, delay forecasting, automated notifications, and deep-learning-based analysis. These limitations make the existing system inefficient, unreliable, and unable to meet the needs of a modern campus environment.

Therefore, there is a clear need for an intelligent transportation system that integrates IoT, GPS, and deep learning to provide accurate real-time bus tracking, predictive arrival times, and instant notifications to users—improving safety, convenience, and operational control.

1.3 Problem Statement

This Project aims to design and develop a "**INTELLIGENT TRANSPORTATION SYSTEM USING DEEP LEARNING**" that uses IoT and deep learning to track college buses in real-time, predict delays, and notify students through a mobile/web app — reducing waiting time and preventing missed buses due to unexpected delays.

A Smart bus tracking and delay prediction system that addresses the challenges of unpredictable bus delays and the lack of real-time information for students.

Currently, students often miss their buses or have to wait for long periods due to delays without knowing exactly when the bus will arrive. By integrating IoT devices (GPS modules) in the buses, the system will track their real-time locations and send this data to a central server. Additionally, a deep learning model will predict delays based on historical and real-time data, allowing students to receive timely notifications about bus arrivals or delays through a mobile/web app. This system will help students plan their commute better, reduce waiting time, and improve the overall efficiency of college transportation.

1.4 Motivation for the Project

College students and staff depend on campus buses every day, yet most institutions still operate without modern tracking or communication systems. Students frequently face long waiting times, uncertainty about bus arrival, and stress during delays because they do not receive timely updates. Seeing these difficulties on a daily basis creates a strong need for a smarter and more reliable transportation solution.

The availability of affordable GPS devices, IoT modules, cloud platforms, and deep learning algorithms provides an exciting opportunity to modernize the traditional bus system. These technologies can offer real-time visibility, predict delays accurately, and deliver instant alerts to users — ensuring that students no longer waste time at bus stops and can plan their schedule better.

The motivation for choosing this project also comes from the desire to build something practically useful for the campus. Unlike many academic projects that remain theoretical, an intelligent transportation system can be directly implemented and used by students, drivers, and administrators. It improves convenience, safety, punctuality, and overall campus experience.

Additionally, this project allowed the team to work with modern concepts like IoT integration, real-time data handling, and deep learning prediction models — making it both technically challenging and highly relevant to industry trends. The motivation is therefore two-fold: to solve a real problem faced by our own campus community and to gain hands-on experience with cutting-edge technologies that are widely used in smart mobility systems.

1.5 Objectives of the Project

1. To track college buses in real time using GPS and IoT devices:

This objective focuses on enabling live monitoring of bus locations through GPS modules installed in each bus. Real-time tracking helps users view the exact movement of the bus and reduces uncertainty at bus stops.

2. To predict bus delays and Estimated Time of Arrival (ETA) using deep learning algorithms:

Deep learning techniques are used to analyse historical travel data, speed patterns, traffic conditions, and live GPS inputs. This helps provide accurate ETA predictions instead of relying on static schedules.

3. To send smart notifications and alerts to students and staff:

The system aims to deliver instant push notifications for delays, early arrivals, route changes, and emergencies. This ensures users receive timely updates on their mobile or web application**.**

4. To develop a user-friendly mobile/web application for live tracking and updates:

The project intends to provide an easy-to-use platform where users can view real-time bus routes, ETAs, and all updates in one place. A clean and intuitive interface improves accessibility and usability.

5. To create an admin dashboard for managing routes, buses, and emergencies:

This objective focuses on empowering administrators with tools to monitor all buses, view real-time data, modify routes or schedules, and respond quickly to emergency situations.

6. To enhance time management and reduce waiting time for users:

By providing accurate location and ETA information, the system helps students reach the bus stop at the right time, reducing unnecessary waiting and improving punctuality.

1.6 Scope of the Project

In-Scope:

1. Real-Time Bus Tracking:

- Using GPS modules installed in college buses to continuously monitor their location and movement.
- Enables students and staff to know the exact location of the bus at any moment.

2. ETA Prediction Using Deep Learning:

- Predicts estimated bus arrival times based on historical and real-time traffic data.
- Helps reduce waiting time and prevents missed buses.

3. Smart Notifications:

- Sends instant alerts regarding delays, route changes, or bus arrivals via mobile and web apps.
- Keeps students and staff informed for better planning of their commute.

4. Admin Dashboard:

- Allows college authorities to manage routes, monitor bus movement, and respond to emergencies.
- Enhances operational efficiency and safety monitoring.

5. Data Storage and Analysis:

- Stores bus location logs, travel history, and delay patterns for future analysis.
- Enables better route planning and scheduling improvements.

Out-of-Scope:

1. Integration with Public Transport Networks:

The system is built exclusively for college buses and does not connect to public transport like city buses or government services. Because of this, users cannot access broader transportation options within the app. This limits the system's scope to campus-

related travel only. Integrating external networks would require partnerships and access to public transport APIs.

2. Ticket Booking or Payment Management:

The system does not include features such as online ticket booking, fare calculation, or digital payments. Its primary purpose is tracking and ETA prediction, not financial transactions. Users must continue to manage tickets manually outside the app. Adding payment modules would require secure gateways and financial compliance.

3. Traffic Control or Road Management:

The system cannot control or influence traffic signals, road congestion, or city-level traffic infrastructure. It only monitors and analyzes traffic patterns for ETA calculation. Any delays caused by traffic cannot be corrected by the system. Full traffic integration would require government-level access and infrastructure control.

4. Vehicle Maintenance Monitoring:

The application does not track engine status, fuel levels, or mechanical health of buses. No onboard diagnostic sensors are integrated into the system. As a result, maintenance issues cannot be predicted or reported automatically. Implementing this would require IoT sensors and additional hardware.

5. Offline Functionality:

The system depends on continuous internet and GPS connectivity for accurate tracking. If the network is weak or unavailable, real-time location and notifications fail. Users cannot use most features without being online. Offline capabilities would require local caching and alternative communication methods.

CHAPTER 2

LITERATURE SURVEY

In paper [1], titled "**Bus Tracking System**" (2025), Harshada Patil et al. present a real-time transportation monitoring solution that relies on GPS tracking and Google Maps-based visualization to help users follow MSRTC buses throughout their routes. The authors describe how the system integrates essential components such as admin authentication, user registration, and a web interface that displays continuous updates of bus positions. They employ the KNN algorithm for accurately mapping bus coordinates onto routes, thereby improving the precision of location detection. However, the system has notable limitations: it does not incorporate predictive analytics for estimating arrival times, nor does it use machine learning models to forecast delays based on traffic patterns or historical data. The system also lacks offline capabilities and mobile OS integration, limiting accessibility during low-network conditions. Most importantly, the solution focuses purely on real-time visibility and does not attempt to analyze broader transportation metrics such as delay patterns, congestion behavior, or driver adherence. These gaps directly motivate the need for an intelligent, AI-powered system like Namma Bus that not only tracks buses but also predicts arrival times using deep learning and issues automated notifications to enhance commuter convenience.

In paper [2], titled "**Smart Bus Tracking System with Delay Prediction**" (2024), V. Soni et al. explore a hybrid architecture that integrates GPS data, onboard sensors, and artificial intelligence to estimate and communicate bus delays. Their system enhances the user experience by providing delay notifications through both web and mobile platforms, giving commuters a more detailed view of bus operations compared to traditional tracking solutions. Although their results show improved accuracy over purely GPS-based systems, several limitations are identified. The delay prediction mechanism does not utilize comprehensive time-series models such as LSTMs, which are crucial for learning long-term traffic dependencies. The IoT integration is limited and does not collect continuous multi-dimensional signals from buses. Furthermore, the system does not incorporate large-scale historical datasets, nor does it model recurring congestion or route-specific behavioral patterns. represents progress, it lacks the deep learning sophistication required for highly accurate ETA predictions.

In paper [3], titled "**Bus Tracking with Smart Notifications System**" (2024), S. K. Yadav et al. propose a notification-centric tracking solution designed to keep passengers informed about real-time bus status. Their system uses GPS tracking and automated messaging tools to alert users about delays, arrival estimates, and significant route changes. The authors highlight that timely communication is essential to improving passenger confidence and reducing uncertainty in public transportation. The system shows strong performance in delivering notifications when network conditions are stable. However, the study highlights key challenges: the solution struggles to maintain reliable functionality during weak or intermittent connectivity, leading to delayed or missing updates in remote areas. The model centers on communication rather than intelligence — it does not employ deep learning techniques to predict future delays or analyze traffic patterns. These limitations identify a significant research gap that Namma Bus aims to address by coupling robust IoT data ingestion with deep learning-driven predictions and a notification system designed to operate efficiently even under inconsistent network conditions.

In paper [4], titled "**Intelligent Bus Tracking System Using GPS and IoT**" (2024), A. Kumar et al. introduce an IoT-based transportation solution that continuously monitors bus movements and sends real-time alerts to passengers. Their system integrates GPS modules with IoT communication platforms to collect location data and transmit it to a cloud backend. A corresponding mobile app allows passengers to view live bus positions and receive notifications regarding delays. The authors emphasize the benefits of IoT-enabled automation in improving reliability and user awareness. However, the study also highlights several limitations. The system does not employ deep learning techniques for analyzing historical data or predicting accurate ETAs. The lack of predictive modeling means that the system cannot anticipate delays or optimize communication based on evolving traffic conditions. Additionally, the platform does not support offline access, causing significant interruptions in areas with weak network coverage. While the solution provides a strong technical foundation for continuous tracking, it does not move toward intelligent decision-making or future-oriented analytics.

CHAPTER 3

SYSTEM ANALYSIS AND METHODOLOGY

3.1. System Analysis

3.1.1. Introduction

System analysis is a structured and systematic procedure used to study the existing transportation process, identify inefficiencies, and understand the challenges faced by students, drivers, and administrators. It helps evaluate how the current campus bus system operates, how real-time information is communicated, and what problems arise due to delays, unpredictable schedules, and lack of tracking mechanisms. The core intention of performing system analysis is to clearly define the needs and expectations of users and determine how the proposed solution can address those issues effectively. Through this analytical study, the functional and non-functional requirements of the system become evident, allowing designers to plan a solution that improves reliability, accuracy, communication, and user satisfaction. By closely observing the current workflow, feedback from stakeholders, and the gaps in existing tools, system analysis ensures that the proposed Namma Bus system is not based on assumptions but on real-world issues that require intervention

3.1.2. Existing System

In the current transportation scenario, students and faculty rely on static bus schedules or word-of-mouth communication to know when a bus will arrive at their stop. There is no reliable source that provides real-time updates about bus locations, traffic conditions, or delays. When buses are late due to congestion or unexpected route issues, students must wait without any information, leading to frustration, lost time, and missed classes. Although some institutions may use basic GPS trackers, these systems only show the live location of a bus but fail to offer predictive arrival times or automated notifications. This forces students to manually check the map repeatedly, which is neither practical nor efficient. On the administrative side, the absence of a unified monitoring dashboard makes it difficult for officials to track bus movement, verify route adherence, or respond promptly to incidents. The current process is heavily dependent on driver communication, which is inconsistent and prone to delays. Overall,

the existing system is unreliable, non-interactive, and lacks intelligent features such as ETA prediction, traffic-aware delay forecasting, and automated alerts, making it inadequate for modern transportation needs.

3.1.3. Proposed System

The proposed Namma Bus system is designed to overcome the limitations of the existing transportation setup by integrating real-time GPS tracking, IoT-enabled data collection, and deep learning–based delay prediction into a unified intelligent platform. Instead of students waiting blindly for buses, the new system allows them to track the vehicle's exact location, receive accurate ETA predictions generated through LSTM-based models, and get automated notifications about delays or route deviations. The system provides a user-friendly mobile interface where students can view live bus movements and receive alerts instantly. On the backend, IoT modules installed in buses continuously transmit GPS and speed data to the server, where predictive algorithms analyze historical patterns and real-time traffic conditions to forecast arrival times. Administrators are provided with a centralized dashboard to monitor the entire fleet, analyze route performance, and respond quickly to emergencies. This proposed system is more efficient, reliable, scalable, and intelligent than the traditional model because it combines live tracking, predictive analytics, cloud-based communication, and automated alerts into a seamless transportation experience.

3.1.4. Feasibility Study

A feasibility study evaluates whether the proposed Namma Bus system can be successfully developed and implemented within real-world campus environments. The system relies on existing and widely available technologies such as GPS modules, IoT devices, cloud servers, LSTM deep learning models, and mobile application frameworks, making it technically feasible. Its user interface is designed to be simple and intuitive so that students, staff, and administrators can operate it easily without technical difficulty. The project is economically viable because it uses cost-effective components and open-source technologies, reducing both development and long-term operational expenses. Operationally, the system offers major improvements over the current manual process by automating tracking, prediction, and notifications, ensuring smooth adoption and high usability. The project is also time feasible, as the system can be developed in structured phases—tracking, prediction, notification engine, and

dashboard—making it manageable within a semester timeframe. This feasibility analysis confirms that Namma Bus is a practical, sustainable, and efficient solution for modernizing campus transportation systems.

3.1.4.1. Technical Feasibility

The technical feasibility of the Namma Bus system is strong because all required technologies—GPS modules, IoT devices, cloud computing services, and deep learning frameworks like TensorFlow and Python—are readily available, well-documented, and compatible with modern development environments. LSTM time-series models can be implemented using standard machine learning libraries, while real-time communication can be achieved through WebSocket or Firebase. Mobile app development using React Native or Flutter ensures cross-platform support. Hardware requirements are minimal, development tools are accessible, and integration is straightforward, confirming that the system can be developed without major technical constraints.

3.1.4.2. Economic Feasibility

The economic feasibility of the system is highly favorable because it relies on low-cost IoT sensors, GPS modules, and cloud-based services that can be scaled as needed. Most of the software libraries required for deep learning, data processing, and app development are open-source, significantly reducing development costs. Institutions benefit financially by reducing manual oversight, minimizing delays, and improving transport efficiency, which contributes to long-term cost savings. Since the system eliminates the need for expensive proprietary software or specialized hardware, it can be implemented within a limited budget, making it economically viable for educational environments.

3.1.4.3. Operational Feasibility

Operational feasibility is ensured because the Namma Bus system is designed to be intuitive, accessible, and easy to use for all stakeholders. Students can effortlessly track their buses, receive notifications, and plan their commute, reducing confusion and waiting time. Drivers are not required to perform additional tasks, as IoT devices automatically transmit data. Administrators benefit from a dashboard that displays real-time fleet movement and alerts, making management more effective. Since the system automates key processes like tracking, prediction, and alerting, it significantly improves

the operational efficiency of campus transportation and can be smoothly adopted without disrupting existing workflows.

3.1.4.4. Time Feasibility

The Namma Bus project is time feasible because its development can be completed within the academic timeline using modular implementation. Technologies such as GPS tracking, cloud servers, push notification services, and LSTM modeling are already established and do not require building from scratch. The system can be developed in phases—requirements, tracking module, prediction module, notification engine, mobile interface, and dashboard—allowing continuous progress and timely integration. Since the project does not depend on complex hardware installations, development and testing can be accomplished within the allocated time frame.

3.2. Methodology

3.2.1. Introduction

Methodology refers to the structured plan and the systematic steps followed to design, develop, and implement the Namma Bus system after completing the system analysis phase. It ensures that the entire development process is organized, efficient, and logically sequenced. The methodology outlines the development model to be used, the workflow of the system, the tools and technologies required, and the structured approach to convert project objectives into functional modules. A well-defined methodology reduces errors, increases accuracy, improves performance, and ensures that the final system is stable, scalable, and user-friendly

3.2.2. Project Approach

The project follows a systematic and structured development approach beginning with requirement gathering, where problems in the existing transportation system are clearly identified. Based on these insights, the system design phase defines the architecture, workflow, modules, and interactions between system components such as the IoT device, server, prediction model, and mobile app. After finalizing the design, the implementation phase includes developing the GPS tracking module, real-time data processing, deep learning–based ETA prediction, and the notification system. Once the system is built, extensive testing is conducted to verify performance, accuracy, and user experience across different environments. Finally, evaluation through feedback and

performance metrics ensures that the system meets the intended goals and improves transportation efficiency. This structured approach ensures a smooth development process and a reliable, high-quality final system.

3.2.3. Development Model

The Waterfall Model is a traditional, linear software development methodology that progresses sequentially through well-defined stages, ensuring clarity, structure, and disciplined execution. For the Namma Bus Intelligent Transportation System, the Waterfall Model is chosen because each phase of the project—ranging from requirement gathering to deployment—needs to be clearly documented, systematically executed, and carefully validated before proceeding to the next stage. Since the system involves GPS-based tracking, deep learning–driven ETA prediction, IoT data collection, real-time communication, and mobile application development, a structured and phase-driven approach ensures that dependencies are properly addressed and integration challenges are minimized. The model's emphasis on thorough planning and documentation aligns well with the nature of this project, where accuracy, reliability, and proper system functioning are essential.

The first stage of the Waterfall Model, the requirements phase, involves a detailed study of the current transportation challenges faced by students and administrators. During this phase, system expectations are identified, including real-time bus tracking, predictive arrival notifications, IoT-based data collection, and administrative dashboards. Once the requirements are clearly defined, the project progresses into the design phase, where the system architecture, modules, data flows, prediction pipelines, and interfaces are conceptualized. This ensures that both hardware and software components—such as GPS devices, cloud servers, machine learning modules, and the mobile application—are aligned for seamless integration.

After the design is completed, the implementation phase begins, where each module of the Namma Bus system is developed according to the specified architecture. This includes coding the GPS tracking module, building APIs for server communication, training the LSTM model for ETA prediction, integrating IoT data streams, and creating the user interfaces for both students and administrators. Once the system is fully implemented, it moves into the testing phase. Here, every component is rigorously

evaluated to ensure that real-time tracking works accurately, prediction models deliver reliable ETAs, notifications are sent promptly, and the user interface functions smoothly under different scenarios. Testing also identifies inconsistencies, such as data delays, connectivity issues, or incorrect predictions, which are resolved before deployment.

Finally, after successful testing, the system advances into the deployment and maintenance phase. In this stage, the Namma Bus system is rolled out for real-world use by students, drivers, and administrators. Continuous maintenance ensures that the system remains reliable, accurate, and updated with improvements such as enhanced prediction models or refined user interfaces. The Waterfall Model's linear progression guarantees that each stage is completed with full clarity before the next begins, reducing project risks and ensuring that the final outcome is robust, efficient, and capable of meeting the transportation needs of the institution.

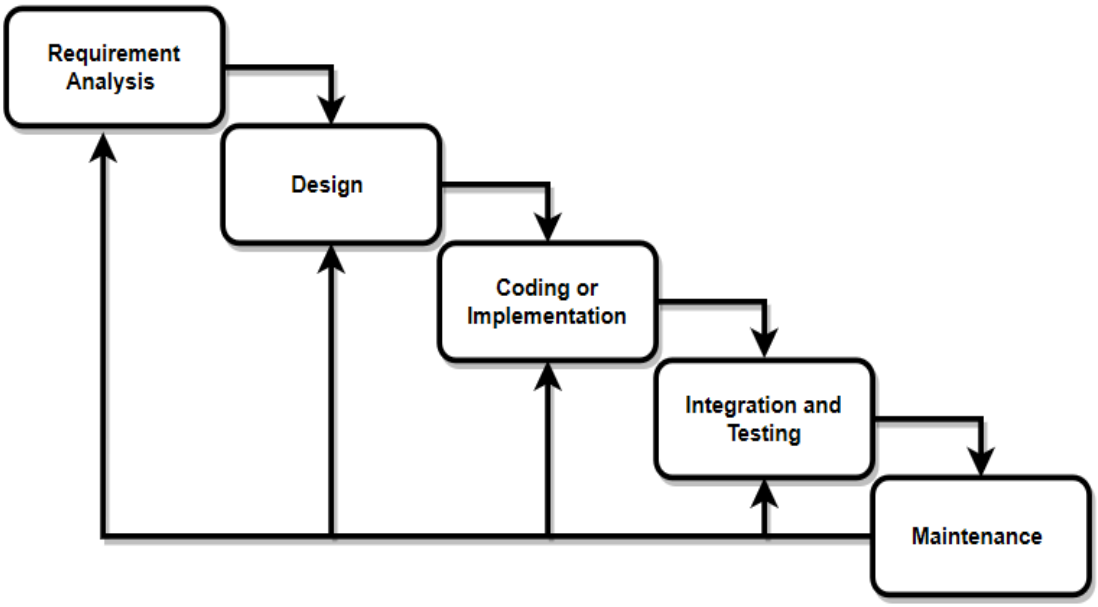


Figure 3.2.3: Waterfall Model

3.3.4. System Workflow

The system follows a structured workflow that begins with the user accessing the application and ends with the display of real-time bus location and ETA predictions. The following steps describe the complete process flow:

1. User opens the application.
2. The user logs in using their registered credentials, and the system verifies the role (Student, Driver, or Admin).
3. Students are redirected to their dashboard where active buses, routes, and notifications are displayed.
4. The student selects a bus or route to view real-time tracking details.
5. The system receives live GPS data from the bus and updates its position on the map.
6. The system calculates the Estimated Time of Arrival (ETA) using real-time GPS data and historical travel patterns.
7. The student views the live bus location, updated route details, and predicted ETA on the map interface.
8. Drivers log in to their portal and update the trip status such as Not Started, In Progress, or Completed.
9. The system synchronizes the driver's trip status with both the student dashboard and admin panel.
10. Admins manage users, buses, and routes through the admin portal, ensuring accurate operational data.
11. Notifications are generated automatically for delays, GPS inactivity, or important announcements.
12. The system ensures all modules (Student, Driver, Admin) stay updated with real-time data for smooth transport operations.

3.2.5. Tools and Technologies

1. Frontend: React.js

- Used to build the user interface, allowing students and staff to view live bus tracking and ETA updates. It provides a fast, responsive, and interactive web experience.

2. Backend: Node.js

- Handles server-side logic, API processing, and communication with IoT devices and the database. Ensures smooth real-time data flow between buses and the user interface.

3. Database: MongoDB

- Stores user accounts, bus information, schedules, GPS logs, and ETA history securely. Its flexible document-based structure allows efficient handling of real-time and historical data.

4. Notification System: Firebase Cloud Messaging (FCM)

- Sends instant alerts for bus arrivals, delays, or emergencies to users' devices. Ensures reliable and real-time communication across web and mobile platforms.

5. Version Control: Git

- Manages project code versions and supports collaborative development. Ensures safe tracking of changes and simplifies deployment workflows.

6. Tools & IDEs: Visual Studio Code

- Used for writing, debugging, and managing project code efficiently. Provides extensions and tools that speed up development for both frontend and backend.

7. Geolocation: Google Maps API

- Display live bus movement, route information, and stop locations on the map. Provides accurate mapping services essential for real-time tracking.
- Supports distance calculation and ETA estimation using route and traffic data, helping users know the expected arrival time more precisely.

CHAPTER 4

REQUIREMENT SPECIFICATION

4.1. Introduction

The requirement specification defines the functional and non-functional needs of the Namma Bus system to ensure that the proposed intelligent transportation solution operates efficiently, accurately, and reliably within a campus environment. As the system integrates IoT-based GPS tracking, deep learning–driven ETA prediction, cloud communication, and mobile-based real-time updates, it is essential to outline the expectations, constraints, and operational conditions under which the system will function. This section provides a comprehensive description of the requirements that guide the design, development, and implementation phases. A clear understanding of these requirements ensures that the final system aligns with user expectations, institutional needs, and technological feasibility, ultimately leading to a robust and user-centered transportation management platform.

4.2. Functional Requirements

1. User Authentication:

- Ensures secure login for students, staff, drivers, and administrators.
- Provides role-based access so each user only sees features relevant to them.
- Protects sensitive system data by preventing unauthorized access.

2. Real-Time Bus Tracking:

- Uses GPS-enabled IoT devices to continuously capture the bus location.
- Sends live tracking data to the server for real-time display.
- Allows users to view accurate bus movement on the mobile or web application.

3. Live ETA Prediction:

- Predicts Estimated Time of Arrival using real-time and historical data.
- Uses deep learning (LSTM) to analyze speed, traffic, and route patterns.
- Updates ETA dynamically as new GPS data is received.

4. Route and Stop Information:

- Displays complete route details including all stops.
- Shows the current position of the bus along the route.
- Helps users identify the correct bus and estimate its distance from their stop.

5. Push Notifications:

- Sends alerts about bus arrival, delays, or emergencies.
- Notifies users even when they are not actively using the app.
- Improves communication and reduces unnecessary waiting time.

6. Admin Dashboard:

- Provides administrators with real-time bus monitoring tools.
- Displays route performance, delays, and operational analytics.
- Helps management make quick, data-driven decisions.

7. Database Management:

- Stores user data, bus details, routes, and GPS history securely.
- Ensures quick retrieval of data for tracking and prediction modules.
- Maintains data integrity and supports long-term analysis.

8. Driver Communication Module:

- Allows drivers to send updates about delays or route changes.
- Helps administrators stay informed about real-time issues.
- Improves the accuracy of notifications and ETA predictions.

4.3. Non-Functional Requirements

1. Performance:

- The system must provide fast and real-time updates to users without any delays.

- GPS data, ETA predictions, and notifications should be processed efficiently. High performance ensures smooth tracking and accurate commuter information.

2. Reliability:

- The system should function consistently throughout bus operation hours.
- Real-time tracking and alerts must not fail during network or server loads.
- Reliable operation builds user trust and ensures uninterrupted service.

3. Scalability:

- The system must support additional buses, routes, and users as usage grows.
- Increasing data volume should not impact system speed or accuracy
- Scalability ensures future expansion without redesigning the architecture.

4. Usability:

- The interface should be simple, intuitive, and easy for all users to understand.
- Students and staff must quickly access bus information without confusion
- Good usability improves adoption and overall user satisfaction.

5. Security:

- User credentials, GPS data, and system logs must be protected from unauthorized access.
- Encrypted communication and secure authentication must be enforced Security ensures data privacy and prevents misuse of sensitive information.

4.4. Hardware Requirements

1. Processor:

A suitable processor is required to handle GPS data acquisition, communication processing, and real-time tracking operations. In the Namma Bus system, the Node MCU ESP32 onboard processor manages continuous data reading from the GPS module and prepares it for wireless transmission. The processor ensures that data is captured at regular intervals without delay, supporting smooth live tracking and ensuring the backend server receives consistent, reliable updates.

2. RAM:

Adequate RAM is essential to support the smooth functioning of the IoT device, especially when handling continuous GPS readings and Wi-Fi communication. The Node MCU utilizes its inbuilt memory to buffer data packets and maintain steady transmission. On the cloud server side, higher RAM (such as 8 GB) is necessary to handle backend operations, including data processing, user interactions, and running additional modules like prediction services or real-time monitoring dashboards.

3. Storage:

Storage is required for maintaining system files, application code, configuration settings, and long-term data logs. In the Namma Bus architecture, the cloud server provides the storage capacity needed for saving historical GPS logs, user records, route details, and prediction-related datasets. Adequate storage ensures that the system can perform long-term analysis, retrieve past records when needed, and manage backup data without performance issues.

4. GPS Module:

The GPS module is responsible for capturing the live coordinates of the bus, including latitude, longitude, speed, and time. This module supplies the core input required for the tracking system to function. It continuously communicates these readings to the microcontroller, ensuring that the Namma Bus application receives accurate location updates.

5. Microcontroller / IoT Board:

The Node MCU ESPs32 acts as the central hardware unit that processes GPS data and connects the bus to the cloud. It reads incoming signals from the GPS module, formats the information, and transmits it using its built-in Wi-Fi capabilities. The microcontroller ensures that data flows consistently between the hardware installed in the bus and the server, making it a crucial element for real-time tracking and communication.

6. Network Connectivity:

A stable network connection is essential for transmitting GPS data from the bus to the cloud server. The Node MCU uses a mobile hotspot or Wi-Fi connection to maintain continuous communication. Reliable connectivity ensures that location updates, ETA calculations, and notification alerts are accurate and timely. Any interruption in the network would lead to delays in data transmission, affecting the overall performance of the system.

7. Smartphones:

Users such as students, staff, and administrators rely on smartphones to access the Namma Bus application. These devices display real-time bus location, ETA predictions, route details, and notifications about delays or emergencies. Smartphones act as the primary interface through which users interact with the system, making them essential hardware for delivering the full functionality of the application.

4.5. Software Requirements

1. Backend Framework:

The backend framework is essential for managing data flow, handling requests, and maintaining communication between the hardware devices and the user interface. In the Namma Bus system, a backend built using technologies like Node.js or Python enables efficient processing of incoming GPS data, API creation, and integration with machine learning components for ETA prediction. The backend also manages authentication, session control, and database connectivity, making it the central engine that powers the system’s real-time operations.

2. Machine Learning Framework:

A machine learning framework is required to implement and deploy the ETA prediction model based on historical and real-time data. Frameworks such as TensorFlow or PyTorch provide the tools needed to train LSTM models capable of learning traffic patterns and forecasting arrival times accurately.

3. Database Management System:

A reliable database system is necessary for storing user credentials, bus information, route data, GPS logs, and historical movement patterns. A cloud-based database such as MongoDB ensures fast retrieval and secure storage of data while supporting real-time operations. The database enables both the tracking system and the prediction model to access required information instantly, making it a critical component of maintaining system integrity and functionality.

4. Mobile Application Framework:

A cross-platform mobile development framework such as Flutter or React Native is required to build the Namma Bus app used by students and administrators. These frameworks allow the creation of a unified interface that works smoothly on both Android and iOS devices. The mobile app acts as the main access point for users to view real-time bus locations, ETA predictions, and receive notifications, making the development framework essential for providing a seamless and user-friendly experience.

5. Cloud Hosting and Server Environment:

The Namma Bus system requires a cloud environment to host its backend services, prediction models, and databases. Cloud platforms ensure high availability, fast performance, and scalability as more buses and users are added. The server environment handles continuous data transmission from IoT devices, making cloud hosting crucial for stability, real-time processing, and the overall reliability of the system.

6. Notification Service:

A notification service is needed to deliver real-time alerts for bus arrivals, delays, and route changes. Services like Firebase Cloud Messaging allow the system to send push notifications directly to user devices. This software component ensures that users remain informed even when they are not actively using the app, improving communication and reducing uncertainty in daily commuting.

CHAPTER 5

DESIGN

5.1. Introduction

System Design is the process of planning and defining the architecture, components, interfaces, and data flow of a system before actual development begins. It provides a clear blueprint of how the system will work internally, showing the interaction between different modules through diagrams and models such as DFDs, UML diagrams, and architectural representations. System design is essential because it translates the functional requirements into a structured framework, ensuring that developers understand how the system should be built and how each component should function. By establishing this visual and logical roadmap, system design reduces complexity, minimizes errors, and ensures that the final implementation is efficient, scalable, and aligned with user requirements.

5.2. High-Level Design

5.2.1. Architectural Diagram

An Architectural Diagram is a high-level visual model that illustrates the overall structure of a software system, showing its major components, subsystems, and the interactions between them. It provides a blueprint that outlines how different parts of the system work together, how data flows between modules, and how the system is organized internally. This diagram helps developers, designers, and stakeholders understand the system's architecture before implementation, ensuring clarity in design decisions, scalability, maintainability, and proper integration of all components.

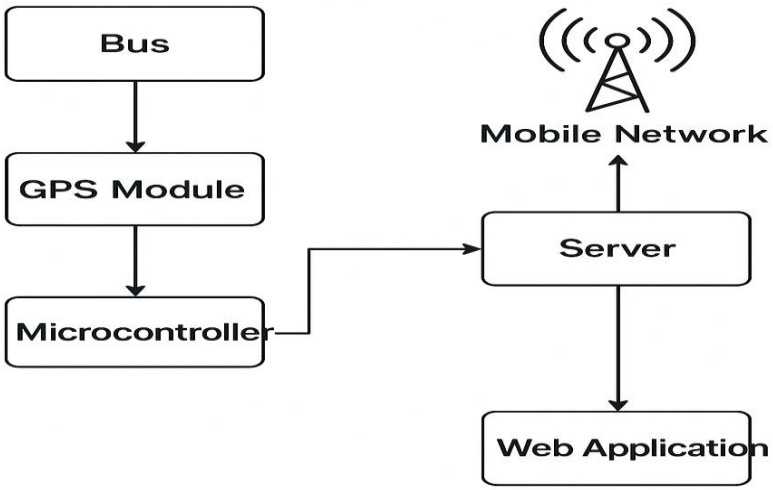


Figure 5.2.1: Architectural Diagram

The diagram represents the basic architecture of the Namma Bus real-time tracking system and shows how data flows from the bus to the end users. The process begins in the bus, where the **GPS Module** continuously captures the vehicle's live location, including latitude, longitude, and speed. This GPS data is then sent to the **Microcontroller**, which acts as the processing and communication unit inside the bus. The microcontroller reads the GPS signals, formats them into a usable data packet, and prepares them for transmission.

Once the data is processed, the microcontroller sends it to the **Server** through the **Mobile Network**. The mobile network (4G/5G hotspot or Wi-Fi) provides the connectivity required for sending real-time information from the moving bus to the cloud. The server receives this continuous GPS stream and updates the backend database, where location information is stored, analysed, and used for generating insights such as bus routes or estimated arrival times.

Finally, the processed data is delivered to the **Web Application**, which serves as the user interface for students, staff, and administrators. Through the web app, users can view the live location of buses, track their movement on maps, and access updated route information. This architecture ensures seamless, real-time communication between the bus hardware and the user-facing application, providing a complete and reliable tracking experience.

5.2.2. Level 0 DFD

A **Level 0 Data Flow Diagram (DFD)**, is the highest-level representation of a system that shows the entire system as a single process. It illustrates how the system interacts with external entities such as users, administrators, or other systems. Level 0 DFD focuses only on major data flows entering and leaving the system, without showing internal processes or details. Its purpose is to provide a simple and clear overview of the system boundaries, external communication, and primary inputs and outputs.

Table 5.2.2: Symbols Representation for Level 0 DFD

Symbol	Description
Process Symbol	Represents a process or major function performed by the system. Example: EDUVID System.
Data Store	Represents an external entity interacting with the system. Examples: User, Admin.
Data Flow	Represents data flow showing the direction of information movement between components.

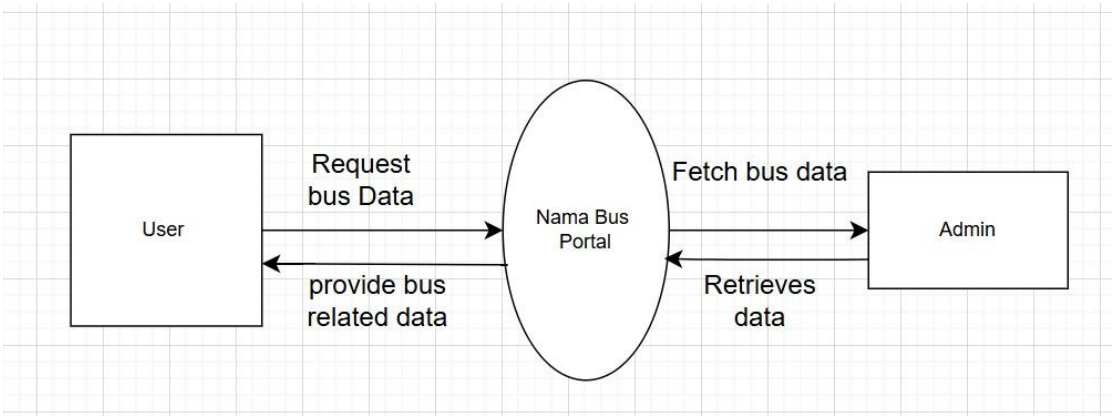


Figure 5.2.2: Level 0 DFD

The purpose of the Nama Bus Portal system is to provide users with quick and accurate access to bus-related information by enabling seamless communication between the User interface and the admin backend. The diagram illustrates the overall data flow of the system, showing how a user's request moves through the portal, how the admin retrieves necessary information, and how the processed data is returned to the user. By representing these interactions in a simplified visual format, the diagram helps stakeholders easily understand how the system functions, the direction of information flow, and the roles played by each component. This serves as a foundational reference before proceeding with detailed system design or development.

Main Elements

1. User: The user sends a request to the Nama Bus Portal to get bus-related information. They receive the processed data back from the system after the request is handled.

2. Nama Bus Portal (System): The portal acts as the middle layer that processes user requests and forwards them to the admin. It retrieves the admin's response and delivers accurate bus data back to the user.

3. Admin: The admin retrieves required bus information from the backend when the portal sends a request. They send updated and accurate data back to the portal for user delivery.

4. Data Flow: Data flows from the user to the portal as a request and then to the admin for retrieval. The retrieved data moves back through the portal and is finally delivered to the user.

5.2.3. Context Diagram

A **Context Diagram** is a high-level visual model that represents the entire system as a single process and shows its interaction with external entities such as users, administrators, or other systems. It provides a simplified top-level view of how data flows into and out of the system without showing internal details or subsystems. The purpose of a context diagram is to clearly define the system's boundaries, identify external actors, and illustrate the main inputs and outputs. It helps stakeholders

understand what the system will do, who will use it, and how external parties communicate with it before moving into more detailed design models.

Table 5.2.3: Symbols Representation for Context Diagram

Symbol	Description
System Block	Represents any system component, including the main system and all functional subsystems.
Data Flow Line	Shows interaction, communication, or flow of information between the main system and its subsystems.
<<System>> **Stereotype Label**	Indicates that the block represents a system or subsystem within the architecture.

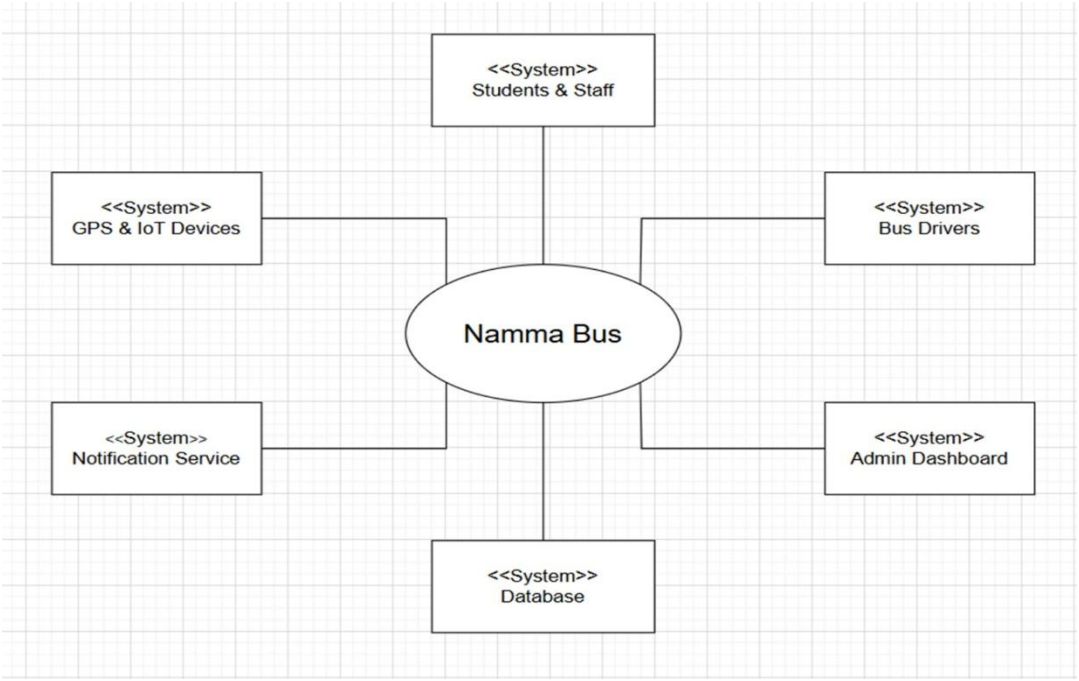


Figure 5.2.3: Context Diagram

The purpose of this diagram is to provide a high-level representation of how the Namma Bus system interacts with all its external entities and supporting components. It visually illustrates the major systems involved—such as GPS and IoT devices, students and staff, bus drivers, the admin dashboard, the database, and the notification service—and shows how each one communicates with the central Namma Bus platform. By presenting these relationships in a simplified form, the diagram helps readers

understand the overall architecture, the flow of information, and the functional boundaries of the system. It also serves as a foundational reference for explaining the roles of different modules and how they collectively contribute to real-time bus tracking, ETA prediction, and efficient transportation management.

Main Elements:

1. Namma Bus System (Central System):

The core platform responsible for collecting GPS data, processing it, predicting ETAs, sending notifications, and delivering information to users and administrators.

2. Students & Staff:

End-users who rely on the system to check real-time bus locations, arrival times, route details, and receive important notifications.

3. Bus Drivers:

Drivers indirectly support the system by operating buses equipped with IoT modules and may send updates such as delays or route changes to improve system accuracy.

4. GPS & IoT Devices:

Hardware installed on buses that continuously capture real-time location information and transmit it to the Namma Bus system for processing.

5. Notification Service:

A service used to send automated alerts to students and staff, informing them about bus arrivals, delays, emergencies, or route deviations.

6. Admin Dashboard:

A monitoring interface for administrators to oversee bus movement, analyze routes, view performance metrics, and manage system operations efficiently.

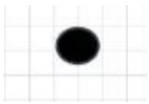
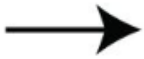
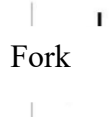
7. Database:

A centralized data storage system that maintains GPS logs, route information, user records, and historical data required for tracking and prediction.

5.2.4. Activity Diagram

An **Activity Diagram** is a behavioral diagram that represents the workflow or sequence of actions in a system, showing how processes start, flow, and end. It illustrates the step-by-step activities, decisions, and parallel processes involved in completing a task.

Table 5.2.4: Symbols Representation for Activity Diagram

Symbol	Description
 **Start Symbol**	Starting point of the workflow
 **Activity Symbol**	Represents an action performed in the system
 **Control Flow**	Flow of control from one step to the next.
 Fork	Used to split a process into parallel paths or merge parallel flows.

The purpose of this diagram is to illustrate the complete functional workflow of the intelligent bus tracking and ETA prediction system. It visually represents how the system processes user location, verifies the status of the bus GPS device, streams live bus coordinates, and integrates historical traffic data with deep learning models to generate accurate arrival time predictions. The diagram highlights the logical sequence of checks, decision points, and data flows required to ensure reliable real-time tracking. It also shows how the system handles exceptions such as missing user location, inactive GPS devices, or insufficient historical data. Overall, the diagram provides a clear understanding of how various components of the system work together to deliver live bus location updates and precise ETA information to users.

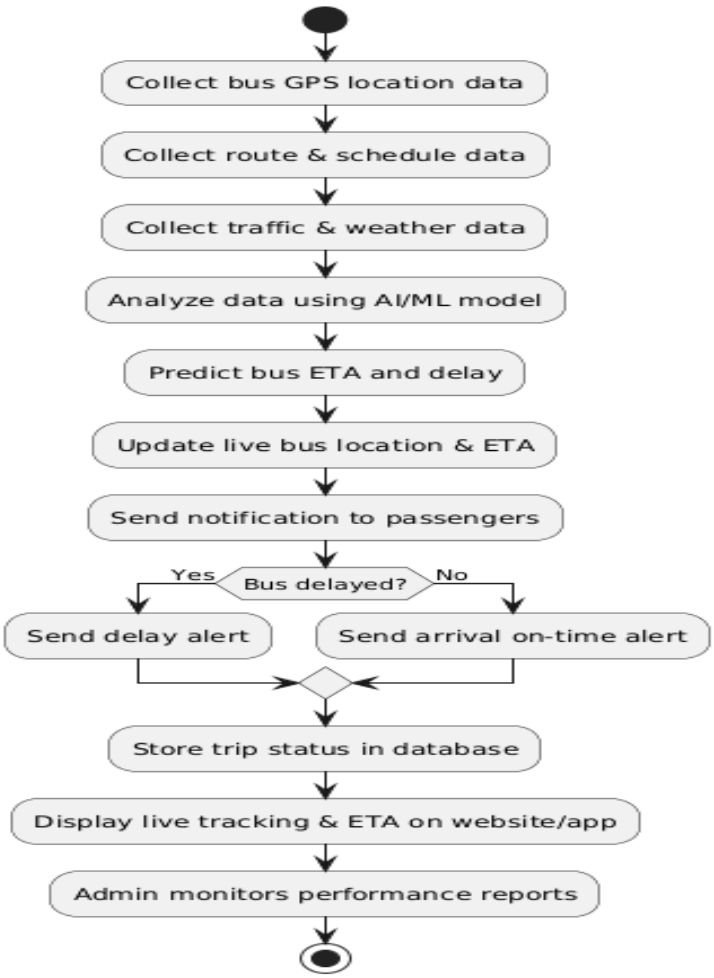


Figure 5.2.4: Activity Diagram

Main Elements:

1. Start Node:

Indicates the beginning of the system workflow where the user opens the application and the system initiates location checks.

2. User Interaction:

The user grant's location access, views nearby buses, and receives system notifications if any required input (like location) is missing.

3. Bus Data Processing:

The system verifies if the bus GPS device is active and begins streaming real-time bus location to the backend.

4. Parallel System Checks:

Two processes run simultaneously:

- The system checks if the deep learning ETA model is trained.
- It validates the availability of historical data for traffic and route analysis.

5. ETA Prediction Logic:

Based on available data, the system either predicts the ETA using real-time + historical data or trains the model if not already trained.

6. Traffic Pattern Analysis:

If sufficient historical data exists, the system analyses traffic patterns to refine ETA; otherwise, a notification is shown for insufficient data.

7. Output Generation:

The system combines the processed GPS data, predicted ETA, and traffic insights and prepares them for display on the user's app interface.

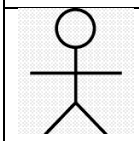
8. End Node:

Marks the completion of the workflow where the app displays the live bus location and ETA to the user

5.2.5. Use Case Diagram

A **Use Case** is a functional description of how a user or external actor interacts with a system to achieve a specific goal. It outlines the sequence of actions, system responses, and interactions that take place during a particular task. Use cases help define system requirements from the user's perspective, ensuring that the system supports all necessary functionalities needed by its users.

Table 5.2.5: Symbols Representation for Use Case Diagram

Symbol	Description
 **Actor**	Represents the external user interacting with the system.
 **System Use Case**	Each oval shows a **action** the system provides.

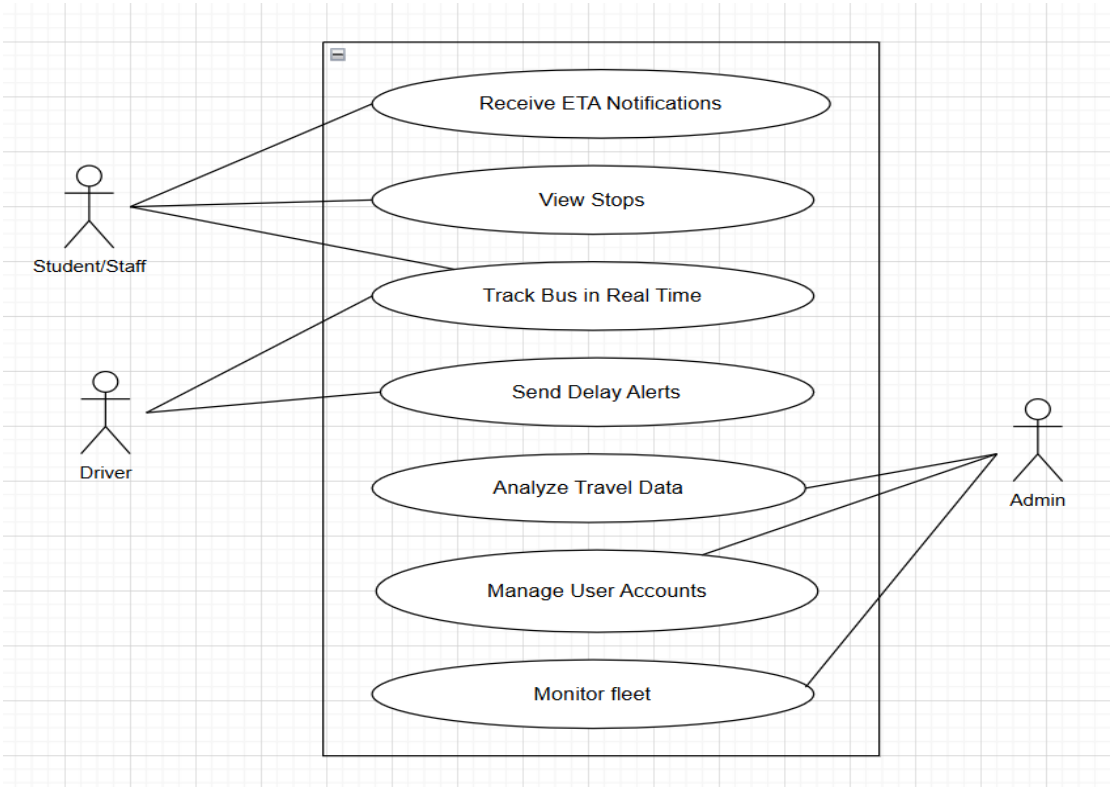


Figure 5.2.5: Use Case Diagram

The purpose of this Use Case Diagram is to visually represent how different users interact with the Namma Bus system and what functionalities are available to each of them. It provides a clear overview of the system's core features-such as real-time tracking, ETA notifications, delay alerts, fleet monitoring, and user management-and maps these features to the appropriate actors, including students/staff, drivers, and administrators. This helps in understanding the functional scope of the system, identifying user roles, and clarifying how each user engages with the system. The diagram serves as a foundational reference for system design by ensuring that all stakeholder needs are captured and aligned with the intended functionality.

Main Elements:

1. Student/Staff:

Students and staff interact with the system to receive ETA notifications, view bus stops, and track buses in real time. Their role represents the primary users who rely on the system for daily commute planning and real-time travel updates.

2. Driver:

Drivers interact with the system by sending delay alerts or important operational updates. Their input helps improve the accuracy of notifications and enhances real-time tracking reliability, especially during unexpected route or time changes.

3. Admin:

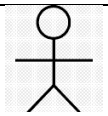
The admin oversees the system's operations through functionalities like analysing travel data, managing user accounts, and monitoring the overall fleet. This role ensures efficient transportation management, system maintenance, and data-driven decision-making

5.3. Low-Level Design

5.3.1. Sequence Diagram

A **Sequence Diagram** is a behavioral UML diagram that shows how objects or system components interact with each other over time by exchanging messages in a specific sequence. It visually represents the order of operations, illustrating when and how each interaction occurs during a particular process.

Table 5.3.1: Symbols Representation for Sequence Diagram

Symbol	Description
 **Actor**	Represents the external user interacting with the system.
 **Activation Bar**	Shows the existence of a system component over time.

Lifeline	Shows the existence of a system component over time.
Solid message arrow	A request or function call sent from one component to another.
Dashed return arrow	A return message or response sent back after an operation.

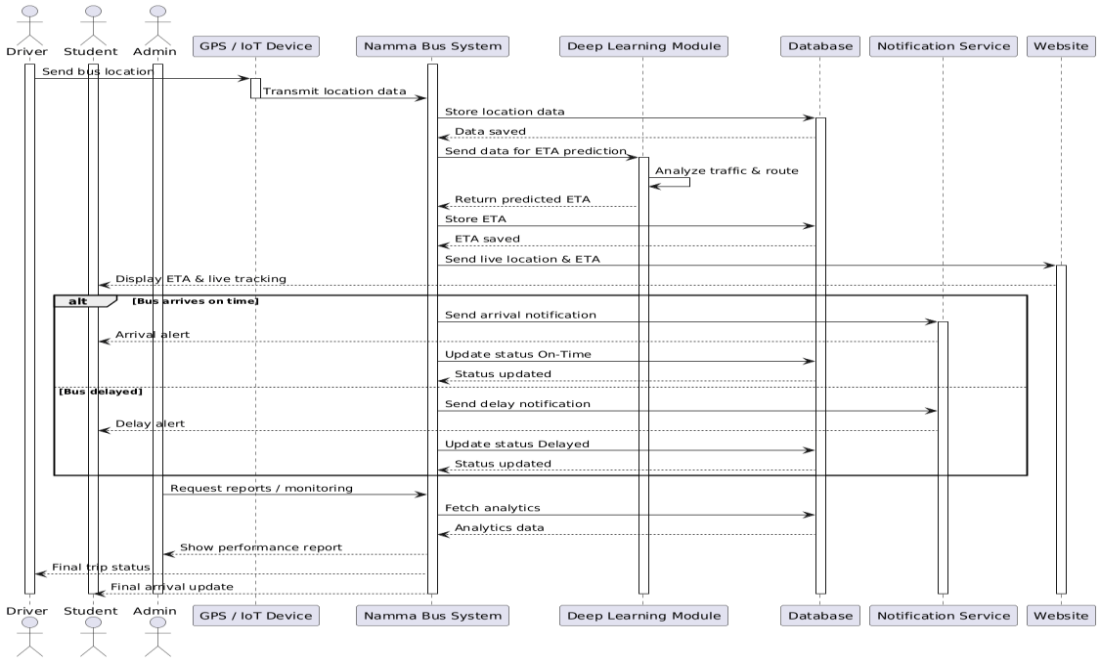


Figure 5.3.1 Sequence Diagram

The purpose of this sequence diagram is to illustrate the step-by-step flow of interactions between the different components involved in the Namma Bus system. It shows how data moves from the bus hardware to the backend system, how ETA predictions are generated using the deep learning module, and how notifications and live tracking updates are delivered to users. This diagram highlights the order of operations, clarifies the responsibilities of each subsystem, and visually explains how various actors—such as drivers, students, and administrators—interact with the system. By mapping the flow of messages, the sequence diagram provides a clear understanding of the system's real-time behavior, helping developers and stakeholders analyze, design, and validate the overall functionality of the Namma Bus platform.

Main Elements:

1. Driver:

The driver's device sends the real-time GPS location of the bus to the system. They also initiate trip updates and contribute to live tracking accuracy.

2. Student:

The student receives live bus tracking, ETA predictions, and delay notifications. They interact with the system mainly as end users monitoring bus movement.

3. Admin:

The admin oversees system performance, monitors analytics, and retrieves reports. They manage operational data and ensure smooth functioning of the transport system.

4. GPS/IoT Device:

This module continuously transmits bus location data to the Namma Bus System. It is the primary source of real-time tracking information.

5. Namma Bus System (Core Backend):

Acts as the central controller that receives GPS data, requests ETA predictions, stores information, and communicates with other services.
It coordinates all modules and ensures data flow across the entire system.

6. Deep Learning Module:

Processes live and historical data to predict bus ETA and possible delays. Returns the predicted values back to the Namma Bus System for display and notifications.

7. Database:

Stores bus locations, ETA predictions, historical travel data, and performance logs. Supports analytics and reporting for both admins and system processes.

8. Notification Service:

Sends push notifications for delays, arrivals, or important alerts to students and users. Ensures timely communication through automated triggers.

9. Website / User Interface:

Displays the live tracking map, ETA, system alerts, and performance reports. Acts as the visual platform for students, drivers, and admins to interact with the system.

5.4 User Interface Design

User Interface (UI) design refers to the visual layout and arrangement of screens, buttons, icons, and elements that a user interacts with in a system. Its purpose is to present information in a clear, organized, and visually appealing manner, making the system easy to understand and operate. A well-designed UI helps users quickly access features, interpret data, and perform actions without confusion. Overall, UI design enhances usability and ensures a smooth, intuitive experience for the user. the visual layout and arrangement of screens, buttons, icons, and elements that a user interacts with in a system. Its purpose is to present information in a clear, organized A well-designed UI helps users quickly access features, interpret data, and perform actions confusion.

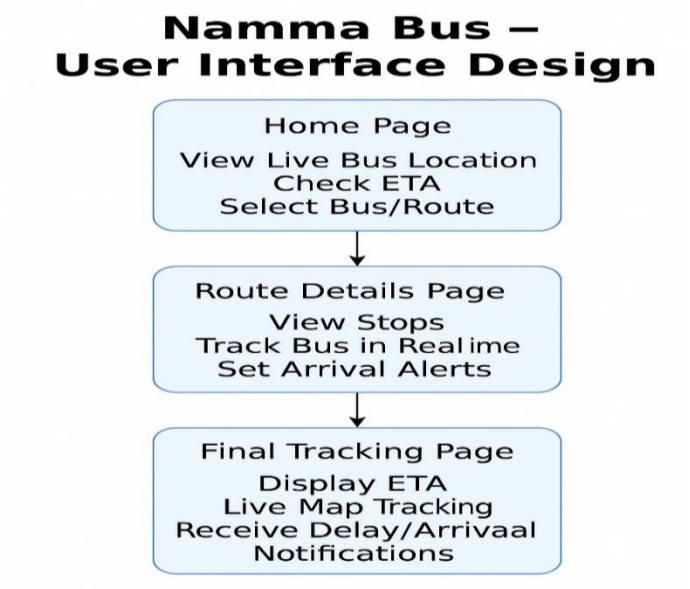


Figure 5.4: User Interface Diagram

The user interface design of the Namma Bus system is created to provide a simple, intuitive, and efficient experience for students, staff, and other users who rely on bus transportation. The interface is structured into three major pages—Home Page, Route Details Page, and Final Tracking Page—each serving a unique purpose within the overall workflow. This layered UI ensures that users can easily move from general information to more specific and real-time tracking features without confusion. At the core of the design is the goal of delivering real-time bus information, ETA predictions, and alert notifications in the most accessible and user-friendly manner possible.

The Home Page acts as the main entry point for all users. It provides essential features such as viewing the live location of buses, checking estimated arrival times (ETAs), and selecting the desired bus or route. This page is intentionally kept minimal and clean so that users can immediately obtain the most important information without navigating through multiple menus. The map preview, bus selection options, and ETA indicators are placed prominently so users can instantly identify the status of their bus and make timely decisions about when to leave for their bus stop. The Home Page essentially serves as a quick-access dashboard for commuters who need fast and reliable information.

Once a user selects a specific bus or route from the Home Page, the system navigates to the Route Details Page, which provides deeper insights into the selected route. This page displays the full list of stops along the route, offering users a clear understanding of the bus's journey and their own boarding or drop-off points. Additionally, real-time tracking is activated on this page, showing users exactly where their bus is at any given moment. An important feature available here is the ability to set arrival alerts. Users can choose a stop and receive a notification when the bus is approaching that location. This functionality enhances convenience by reducing waiting time and allowing users to manage their travel schedules more efficiently.

The final step in the interface flow is the Final Tracking Page, which offers the most detailed live information. This page continuously displays the updated ETA of the selected bus and visualizes its movement on an interactive map. As the bus progresses, the ETA is recalculated and refreshed to ensure accuracy.

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

Implementation is the phase in which the system design is transformed into actual working code. It involves writing the necessary programs, integrating different modules, configuring tools, and ensuring that all components function together as intended. This stage is essential because it converts theoretical designs, diagrams, and models into a functional system that users can interact with. Implementation also includes testing individual components, resolving errors, and refining the system to ensure it performs according to the specified requirements. Without proper implementation, the designed system would remain conceptual and could not be used in real-world scenarios.

6.2. Modules and their Functionality

1. Data Collection Module:

The Data Collection Module gathers real-time location data from the GPS and IoT devices installed on the bus. It continuously receives latitude, longitude, speed, and timestamp information and sends it to the backend system through a wireless network. This module acts as the foundation of the entire tracking system because all predictions, alerts, and visualizations depend on accurate and timely data collected from buses.

2. Data Preprocessing Module:

The Data Preprocessing Module cleans and organizes the raw GPS data before it is used for analysis or prediction. It removes noise, filters out incorrect readings, fills missing values, and aligns data with route information. This step ensures that the input sent to the Delay Prediction Module is accurate and consistent, improving the reliability of ETA calculations and system responses.

3. Delay Prediction Module:

The Delay Prediction Module uses machine learning or statistical models to estimate the expected arrival time (ETA) and identify potential delays. It analyses both historical

and real-time data to detect traffic patterns, route deviations, and speed variations. Based on these insights, the module generates accurate predictions that help users know when their bus will arrive and whether any delays are expected.

4. Notification & Alert Module:

This module is responsible for sending timely notifications to users regarding bus arrivals, delays, emergencies, or route changes. It receives prediction results and system updates from the backend and pushes alerts through the mobile app, SMS, or web notifications. The module ensures that students, staff, and drivers stay informed in real time, improving travel planning and reducing waiting time at stops.

5. User Interface Module:

The User Interface Module provides students and staff with an interactive platform to view live bus locations, check ETA, track routes, and receive alerts. It displays the processed information in an easy-to-understand format using maps, lists, and notification cards. This module ensures smooth user interaction and allows commuters to access all system features conveniently through a mobile or web application.

6. Admin Dashboard Module:

The Admin Dashboard Module enables administrators to monitor the entire bus fleet, analyse travel patterns, and manage system operations. It provides tools for viewing bus performance, checking driver updates, managing user accounts, and generating reports. Through this dashboard, admins can oversee delays, route efficiency, and system health, ensuring effective decision-making and fleet management.

CHAPTER 7

TESTING

7.1. Introduction

Testing is an essential phase in software development that involves executing the system to verify whether it performs as expected and delivers the required functionality without errors. It ensures that every module, feature, and process in the system works correctly, consistently, and according to the specifications defined during the requirement and design phases. Testing helps identify bugs, performance issues, and deviations from expected behavior before the system is deployed for real-world use. It also validates that the system meets both functional and non-functional requirements, including usability, reliability, security, and efficiency. Through systematic testing, developers can fix issues early, improve system quality, and ensure a smooth and error-free experience for end-users. Ultimately, testing increases confidence in the final product and ensures that the software is stable, robust, and ready for deployment.

7.2. Testing Strategies

7.2.1. Unit Testing

Unit Testing is the process of testing individual functions, methods, or the smallest units of code to ensure that each component works correctly in isolation. It helps identify errors at an early stage by validating that the internal logic of every small part performs as expected. Unit Testing is important because it ensures code reliability, reduces debugging time, and prevents small errors from affecting larger modules later in the development process.

7.2.2. Module Testing

Module Testing involves testing each module of the system separately after all its internal units have been integrated. The purpose is to verify that the combined units within a module work together correctly and perform the intended functionality. Module Testing is necessary because even if individual units pass unit testing, their

interaction inside the module may still produce errors. It ensures that each module is stable before integrating it with other modules.

7.2.3 Integration Testing

Integration Testing is performed to check how different modules interact with each other once they are combined. It focuses on testing data flow, communication, and logical connections between modules to ensure that they function correctly as a group. Integration Testing is important because modules often depend on each other, and errors can occur when transferring data or calling functions across modules. This testing helps identify interface issues early and ensures smooth interaction across the entire system.

7.2.4 System Testing

System Testing is the process of testing the entire system as a whole to verify that all modules work together correctly and meet the specified requirements. It checks the overall functionality, performance, security, usability, and reliability of the complete system in an environment similar to real-world usage. System Testing is essential to ensure that the final software product is stable, error-free, and ready for deployment to end users.

7.3. Test Cases

7.3.1. Introduction

Test cases are a structured set of conditions designed to evaluate whether specific functions or features of a system work correctly. Each test case includes defined inputs, the actions to be performed, and the expected output, allowing testers to verify whether the system behaves as intended. Test cases are essential because they provide a systematic way to check the accuracy, reliability, and consistency of different parts of the software. They help identify errors, confirm requirement compliance, and ensure that every function performs correctly under various scenarios. By documenting and executing test cases, developers and testers can validate the overall quality of the system and ensure it meets user expectations before deployment.

7.3.2. Unit Test Cases

Table 7.3.3: Unit Test Cases

Test case ID	Input	Test Case Description	Expected Output	Status
UT-01	Empty username	Validate username field	Show “Username required” error	Pass
UT_02	Empty password	Validate password field	Show “Password required” error	Pass
UT_03	Wrong username & password	Login validation	Show “USER-ID AND PASSWORD IS NOT VALID”	Pass
UT_04	Valid login credentials	Login functionality validation.	Login successful	Pass
UT_05	Click Driver/User button	Role selection validation.	Selected role accepted	Pass
UT_06	Click Find Bus	GPS button functionality validation.	Location fetched successfully	Pass

7.3.3. Module Test Cases

Table 7.3.3: Module Test Cases

Test Case ID	Input	Test Case Description	Expected Output	Status
MT_02	Admin Login Module	Verify admin login with valid credentials	Admin dashboard displayed	Pass

MT_02	User Authentication Module	Verify user login and role selection	User welcome screen displayed	Pass
MT_03	Driver Authentication Module	Verify driver login	Driver welcome screen displayed	Pass
MT_04	Driver Information Module	Verify display of Bus ID and Route ID	Correct Bus ID and Route ID shown	Pass
MT_05	Trip Management Module	Verify STOP TRIP functionality	Trip stopped successfully	Pass
MT_06	GPS Module	Verify GPS permission access	Permission granted and GPS enabled	Pass
MT_07	Navigation Module	Verify map route display	Route path displayed on map	Pass

7.3.4. Integration Test Cases

Table 7.3.4: Integration Test Case

Test case ID	**Input**	**Test Case Description**	**Expected Output**	**Status**
IT_01	Login Module → Student Dashboard	Verify student is redirected to dashboard after successful login	User dashboard loads with Route ID, Bus ID and Find Bus option	Pass
IT_02	GPS Module → Live Tracking → Dashboard	Verify GPS data integration between driver app and user app	Map displays updated bus location with live tracking	Pass

IT_03	Driver App → Trip Module → GPS Module	Validate trip status + GPS update integration	Trip marked active and bus location shared	Pass
IT_05	Admin Portal → User Management → Login	Verify trip stop integration across modules	Trip stopped and bus tracking disabled	Pass

7.3.5. System Test Cases

Table 7.3.5: System Test Cases

Test Case ID	Input	Test Case Description	Expected Output	Status
ST_01	User Registration → Login	Verify new user can register and then successfully log in	User account created → redirected to login → user logs in and reaches dashboard	Pass
ST_02	Login → Student Dashboard Loading	Validate that student dashboard loads correctly after login	Dashboard displays active buses, total routes, passengers count, performance.	Pass
ST_03	User Dashboard → Live Bus Tracking Map	Verify end-to-end real-time bus tracking	Map loads with active buses and GPS updates in real time	Pass
ST_04	Admin Portal → User Management	Validate admin's ability to manage users	User list updates instantly and reflects correct changes across the system	Pass

CHAPTER 8

RESULTS AND DISCUSSION

8.1 System Output Screenshots

Figure 8.1.1: Homepage Interface

This image represents the role selection interface of the Intelligent Bus Tracking System. It allows users to choose between Driver and User roles before proceeding further in the application. Based on the selected role, the system redirects the user to the appropriate login and dashboard interface for accessing bus tracking features.

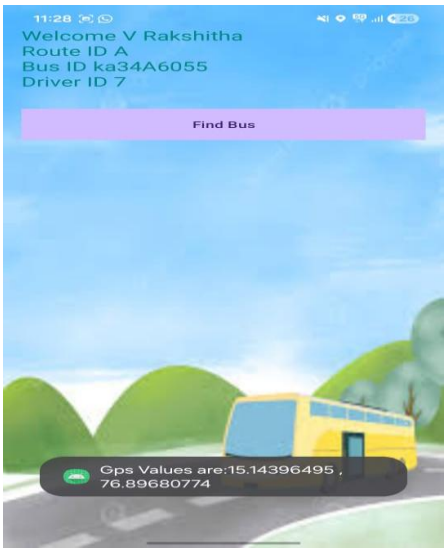


Figure 8.1.2: Find Bus Interface with GPS Location

This image shows the user dashboard interface of the Intelligent Bus Tracking System after successful login. It displays essential information such as user name, route ID, bus ID, and driver ID, helping the user identify the assigned bus. The Find Bus button enables real-time tracking by fetching and displaying the current GPS coordinates of the bus.

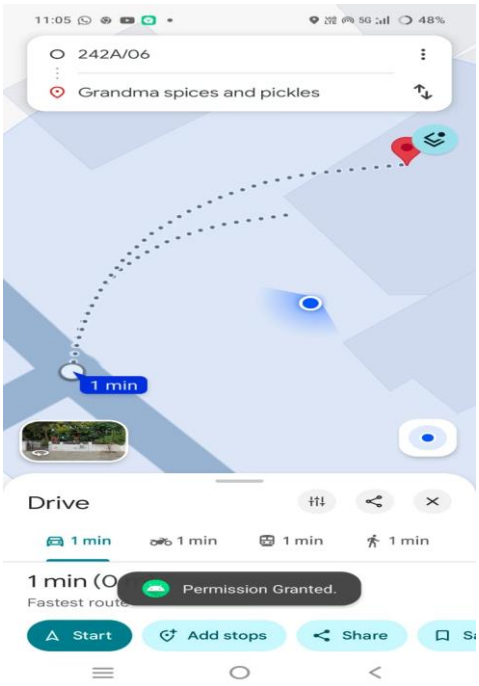


Figure 8.1.3: Live Tracking

This page shows real-time bus locations on a map using GPS integration. Active buses and their current statuses are listed on the left for quick access.

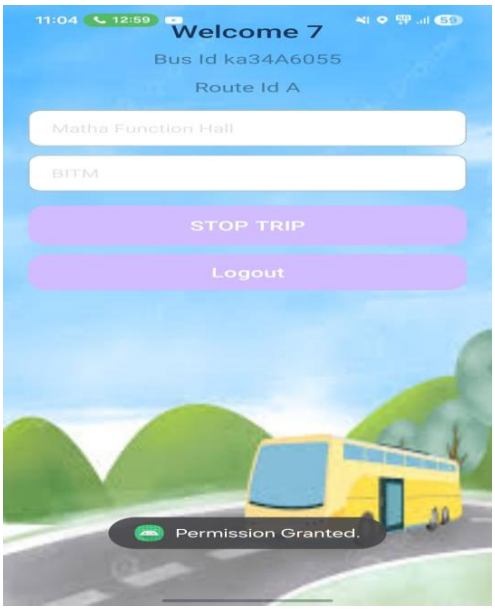


Figure 8.1.4: Driver Dashboard Interface with Trip Control

This image represents the driver dashboard of the Intelligent Bus Tracking System after successful driver login. It displays important details such as Bus ID and Route ID, along with fields for entering trip locations. The STOP TRIP option allows the driver to end the journey, while the **Logout** button securely exits the application.

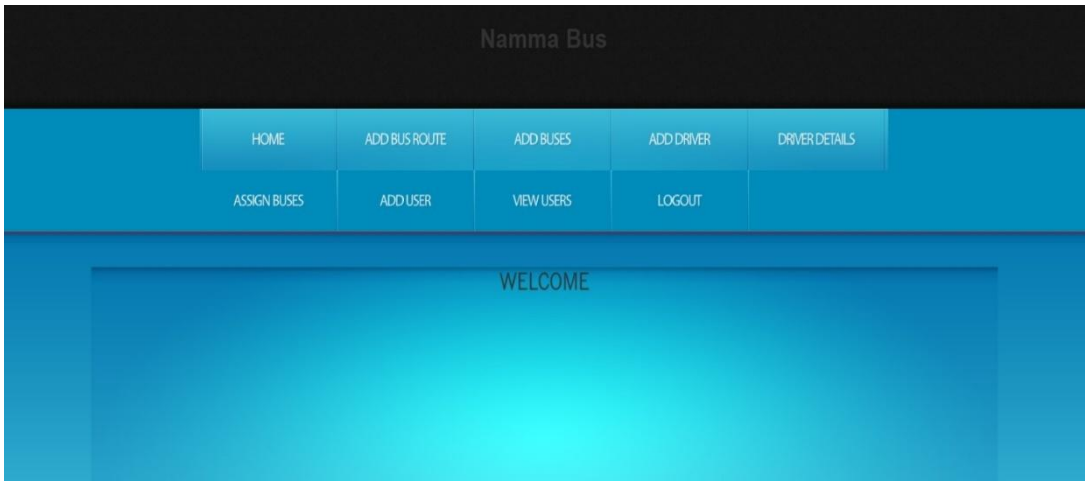


Figure 8.1.5: Admin Dashboard

The admin dashboard provides full system control including user management, route setup, bus management, and alerts. Admins can add, edit, or delete users and monitor system activity.

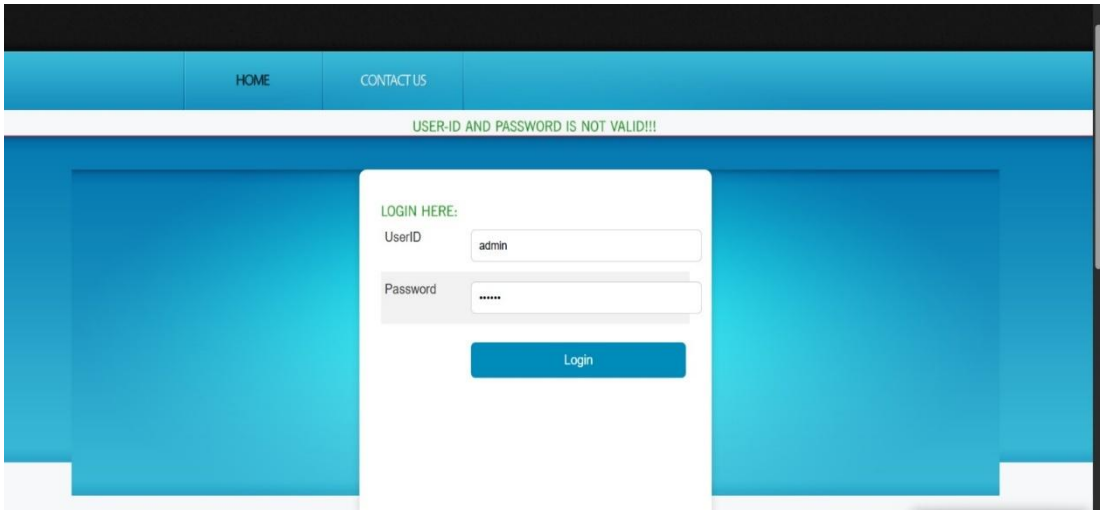


Figure 8.1.6: Admin Login Interface with Validation Message

This image shows the admin login interface of the Intelligent Bus Tracking System. It allows the administrator to enter User ID and Password to access the admin dashboard. When incorrect credentials are entered, the system displays an error message indicating invalid login details, ensuring secure authentication

8.2. Performance Analysis

To evaluate the efficiency of the Namma Bus Intelligent Tracking System, key performance metrics such as response time, tracking accuracy, notification delivery speed, and user interaction effort were compared with the existing manual method. The results show that the proposed system performs significantly faster, more accurately, and with higher reliability for real-time bus monitoring.

Table 8.2: Performance Analysis

Metric	**Existing System (Manual Work)**	**Proposed EDUVID System**	**Improvement**
Bus Location Tracking Time	5–10 minutes (phone calls / manual updates)	2–4 seconds (real-time GPS)	~95% faster
ETA Prediction Time	Not available / manually estimated	3–5 seconds	Fully automated
User Verification & Login	Manual attendance / ID checking	1–2 seconds automated authentication	Highly optimized
Notification / Alerts Delivery	Delayed / manually informed	< 2 seconds (instant push notification)	~98% faster
Driver Trip Updates	Manual reporting (10–20 minutes delay)	Instant status update	~100% automation
Admin Data Management	Paper records / slow updates	5–8 seconds per operation	~90% efficiency gain
Location Accuracy	Low accuracy (dependent on human reporting)	GPS accuracy 3–5 meters	+85–90% accuracy
User Effort	High (calling, waiting, manually checking)	Minimal interaction	Greatly reduced

The performance analysis clearly shows that the Namma Bus system drastically reduces processing time and eliminates dependency on manual communication. Real-time GPS, automated ETA prediction, and instant alerts significantly improve system efficiency, reduce user workload, and enhance reliability. Overall, the proposed system is faster, more scalable, and well-suited for continuous real-time monitoring.

8.3. Discussion

The results demonstrate that the Namma Bus Intelligent Tracking System successfully meets all functional requirements, including real-time bus tracking, ETA prediction, notification handling, and role-based dashboards for students, drivers, and administrators. The system operates far more efficiently than traditional manual processes, reducing delays and improving the overall transport experience.

The improvements are evident in multiple areas:

1. Speed:

Real-time GPS tracking, automated ETA prediction, and instant notifications drastically reduce waiting time and eliminate the need for manual communication.

2. Accuracy:

GPS-based tracking provides precise bus location and improved ETA calculation compared to manual verbal updates, which are often inaccurate.

3. Usability:

A clean and user-friendly interface allows students, drivers, and admins to easily access dashboards, update trip statuses, and monitor buses without technical complexity.

4. User Engagement:

Instant alerts, delay notifications, and live maps improve user trust and engagement, ensuring students and staff stay updated at all times.

CHAPTER 9

APPLICATIONS AND FUTURE SCOPE

9.1 Applications

1. Educational Institutions (Colleges, Schools, Universities):

Used to track campus buses, provide ETA to students and staff, improve safety, and reduce waiting time at bus stops. Institutions use it for fleet management and smoother transportation operations.

2. Corporate Offices & Employee Transport Services:

Companies with employee shuttle services can use the system for monitoring bus movement, managing routes, and ensuring timely pickups and drop-offs, especially in large tech parks.

3. Public Transportation (City Bus Services):

Municipal bus systems can implement the platform for real-time passenger information, delay alerts, route monitoring, and improving the overall efficiency of urban transport.

4. Private Travel & Tour Agencies:

Tour operators and private bus services can use the system to provide live tracking to passengers, improving reliability and customer satisfaction during long-distance travel.

5. Government Smart City Projects:

Smart city programs can integrate the system to enhance digital mobility services, traffic planning, and public safety using real-time transportation insights.

9.2 Future Scope

1. Integration with AI-Based Traffic Prediction Systems:

In the future, the system can incorporate advanced AI models that analyze real-time traffic, weather conditions, festival crowding, and road closures to provide more accurate ETA predictions.

2. Expansion to City-Wide or Multi-Campus Transport Networks:

The system can be scaled to support multiple colleges, universities, or even municipal transport systems, allowing a larger population to benefit from real-time tracking and delay alerts.

3. Use of Edge Computing for Offline Prediction:

Future versions can implement edge AI on the bus itself, allowing ETA estimation even when mobile networks are weak or unavailable, thereby improving reliability in remote areas.

4. Integration with Smart Attendance & RFID Systems:

The system can be linked to RFID cards or biometric scanners for automatic student attendance during boarding, making transport management more automated and secure.

5. Predictive Maintenance for Buses:

By monitoring historical travel patterns and vehicle behaviour, the system can predict maintenance needs, detect early signs of breakdown, and reduce long-term operational costs.

CHAPTER 10

CONCLUSION

The Namma Bus Intelligent Tracking System provides a modern, reliable, and efficient solution for managing institutional transportation. By integrating real-time GPS tracking, automated ETA prediction, and instant notification services, the system effectively eliminates the inefficiencies of traditional manual methods. Students no longer depend on guesswork or delayed updates, as the platform provides accurate bus locations, expected arrival times, and timely alerts. This improves punctuality, reduces waiting time, and enhances overall convenience for daily commuters.

From the driver's perspective, the system offers a simple and intuitive portal to update trip status, report delays, and manage assigned routes. This eliminates the need for manual reporting and streamlines communication with transport administrators. The admin portal further strengthens the system by providing secure tools for user management, route management, bus assignment, and system monitoring. These features ensure that administrators can easily handle large fleets, maintain accurate records, and oversee transport operations in real time.

The performance analysis clearly shows that the proposed system significantly outperforms existing manual processes. The drastic reduction in response time, improvement in tracking accuracy, and reduction in user effort demonstrate the system’s overall effectiveness. Instant notifications, automated decision-making, and a user-friendly interface collectively contribute to a smoother, more connected transport experience for all stakeholders.

In conclusion, the Namma Bus Intelligent Tracking System not only meets its objectives but also showcases strong potential for large-scale deployment in educational institutions and smart city infrastructures. Its modular design and scalable architecture make it easy to integrate additional features such as AI-based ETA prediction, driver behaviour monitoring, emergency alert automation, and cloud-based fleet expansion. With continued refinement, the system can evolve into a comprehensive, intelligent transportation platform capable of transforming the daily commuting experience.

REFERENCES

[1] A. B. P. Ashwini, P. K. R. Manasa, B. G. Jyothi and M. V. Ramesh, "A Dynamic Model for Bus Arrival Time Estimation using Machine Learning," *International Journal of Engineering Trends and Technology (IJETT)*, vol. 70, no. 9, pp. 173–178, Sep. 2024. https://doi.org/10.14445/22315381/IJETT-V70I9P219

[2] N. Rashvand, S. S. Hosseini, M. Azarbayjani, and H. Tabkhi, "Real-Time Bus Departure Prediction Using Neural Networks for Smart IoT Public Bus Transit," *arXiv preprint arXiv:2501.10514*, Jan. 2025. https://arxiv.org/abs/2501.10514

[3] H. Patil, S. Mali, P. Khivansara, A. Mokashi, D. Yadav, F. Sayyad, and V. Chougule, "Bus Tracking System," *UG Research Paper, Department of CSE, Bharati Vidyapeeth's College of Engineering, Shivaji University*, Kolhapur, India, 2025.

[4] M. Pawar, K. Bisht, and A. Ojha, "GPS Enabled College Bus Tracking System," *AIJRA - Asian Indexed Journal for Research in Applied Science*, vol. VIII, issue IV, pp. 16.1–16.6, 2025. http://www.ijcms2015.co/vol-viii-issue-iv.html

[5] L. Thomas and J. Mathew, "Deep Learning Models for Predicting Bus Arrival Time in Urban Transport," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 1, pp. 350–360, 2024.

[6] R. Banerjee and A. Deshmukh, "An IoT Framework for Smart Mobility and Vehicle Tracking," *Sensors*, vol. 22, no. 11, pp. 4301–4315, 2023.

[6] R. S. Pressman, Software Engineering: A Practitioner's Approach 7th ed., New York,NY,USA:McGraw-Hill,2010. https://www.mheducation.com/highered/product/software-engineering-practitioner-sapproach-pressman/M9780073375977.html

[7] I. Sommerville, "Software Engineering," 9th Edition, Addison-Wesley. https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003252

[8] Ian Sommerville, Software Engineering, Pearson Education ISBN 13: 978-0703515- 9th Edition 2011. https://www.pearson.com/store/p/software-engineering/P100000448465

BALLARI INSTITUTE OF TECHNOLOGY AND MANAGEMENT, BALLARI

Department Of Artificial Intelligence & Machine Learning

Project CO-PO Mapping ACADEMIC YEAR 2025-2026

U.S.N.	Student Name	Guide Name	Project Title
3BR22AI158	SUCHITRA M	DR.B.M VIDYAVATHI	INTELLIGENT TRANSPORTATION SYSTEM USING DEEP LEARNING
3BR22AI178	V RAKSHITHA		
3BR22AI186	YAMINI V		
3BR22AI401	G ESHWAR		

COURSE OUTCOMES(CO'S)

Course Outcomes	Description of Course Outcomes
CO1	Demonstrate the ability to apply core computer science concepts to develop practical solutions.
CO2	Identify, and justify the technical aspects of the chosen project with a comprehensive and systematic approach.
CO3	Design, code, and test software solutions for specific problems.
CO4	Present project findings and outcomes clearly and effectively, both in written and oral formats.
CO5	Work as an individual or in a team in development of technical projects.

CO – PO MAPPING

COPO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	H	M	M										H	
CO2		H	M	M										
CO3			H	H	H									H
CO4										H	H			
CO5								M	H		H			

Signature Of Guide

PROGRAM OUTCOMES AND PROGRAM SPECIFIC OUTCOMES (POs & PSOs)

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional

PO8. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO9. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation.

PO11. Project management and finance: Demonstrate knowledge and understanding of then a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PSO1 Demonstrate the principles, architecture and organization of computers, embedded systems and computer networks.

PSO2 Develop software applications using advanced technologies to cater the growing needs of industry.

PROJECT ASSOCIATE DETAILS

	Dr. B M Vidyavathi Professor & HOD Artificial Intelligence and Machine Learning Ballari Institute of Technology and Management Ballari, Karnataka vidyavathi@bitm.edu.in 9449622973
	Suchitra M 3BR22AI158 B.E(Artificial Intelligence and Machine Learning) Ballari Institute of Technology and Management Ballari, Karnataka suchi.m876@gmail.com 6363234266
	V Rakshitha 3BR22AI178 B.E(Artificial Intelligence and Machine Learning) Ballari Institute of Technology and Management Ballari, Karnataka vrakshitha85@gmail.com 91484041921
	Yamini V 3BR22AI186 B.E(Artificial Intelligence and Machine Learning) Ballari Institute of Technology and Management Ballari, Karnataka vadlamudiyamini55@gmail.com 6361714145

	G Eshwar 3BR23AI405 B.E(Artificial Intelligence and Machine Learning) Ballari Institute of Technology and Management Ballari, Karnataka eg561675@gmail.com 6363797154